

1.1 purpose (introduction)

after my first challenge (the resume screener) i came to the conclusion that i understand the concepts well and can find any material i need to code, but the project itself was missing a few key things like a good dataset and a proper pipeline. so for my second challenge im building a model that can detect a fish from an image and tell you what family it belongs to. the idea is for people that do hobby fishing and want to know what they just caught.

1.2 is there really a need for this?

No, there are already plenty of more advanced fish id apps out there. so its not a product i think will sell, the value is more educational.

the stakeholder is the casual fisher, not a researcher or a fishery. the output i expect is: feed it one image, it finds where the fish is (bounding box) and gives back a family + a confidence score.

1.3 pipeline

user inputs image -> model looks at the image -> detects where the fish is -> checks what family the fish belongs to -> outputs the family

1.4 cost of mistakes

no model is 100% accurate so i have to think about what a wrong answer actually costs. two ways it goes wrong.

false positive is when it gives a confident answer thats wrong, or finds a fish where theres none. for my use case the cost is low, the user just gets the wrong name and moves on. but theres one case i cant fully ignore, fish id can touch legal and safety stuff, some families are protected, some have size limits and a few fish are venomous or not safe to eat. so if someone actually believes a wrong label and keeps a protected fish or eats something dodgy, thats a real cost. hence why im keeping this strictly a suggestion tool, it shows confidence and never pretends to be legal or food advice.

false negative is when it just misses the fish and gives nothing back. cost is basically nothing, the user retries with a clearer photo.

the part i actually care about is that these two arent equal. a confident wrong answer is worse than no answer because the user might believe it. so id rather the model stay quiet than guess. thats part of why i use a score threshold of 0.3, and part of why im doing family level and not species level, less detail means less room for a wrong but confident mistake. nothing here is life or death like the cancer example, but that legal/safety edge is why i still count the confident wrong answer as the expensive one.

1.5 analytic approach

"tell me what fish this is from a photo" has to turn into an actual ML problem so i narrowed it down.

1. plain classification (whole image -> family). dropped this. the images have a lot of background noise and the fish is often the same color as the water/rocks, so a classifier on the full image just learns the background instead of the fish.
2. object detection (find the fish first, then label it). this is what i went with, isolating the fish before classifying it is what fixes the noise problem.
3. one stage (YOLO) vs two stage (Faster R-CNN). went two stage, it does region proposals first and then refines them which trades a bit of speed for accuracy, better on the camouflaged fish.
4. backbone, MobileNet vs ResNet50. i train both so my final call is backed by a real comparison and not just a guess.

so the path i landed on is image recognition -> detection (not classification or segmentation) -> two stage Faster R-CNN -> MobileNet vs ResNet.

quick bit of the research that backs the two stage choice: there are basically two types of detectors. one stage detectors like the YOLO series predict the category and bounding box in a single step and are the popular pick for fish detection. two stage detectors like Fast R-CNN, Faster R-CNN and Mask R-CNN use a divide and conquer approach, first generating region proposals and then refining them to improve accuracy. i went with the two stage route for that accuracy on the harder images.

1.6 data availability

normally you can end up starting with no data and building it up, but for me data was actually the easy part. im using the FishNet dataset (<https://github.com/faixan-khan/FishNet/>). its based on an aquatic biological taxonomy with 8 taxonomic classes, 83 orders, 463 families, 3,826 genera, 17,357 species and 94,532 images, and it includes bounding box annotations for detection. so i had a fully labelled detection dataset from day one.

the real problems were the shape and quality of that data, and thats what drove most of my decisions:

- heavily unbalanced, some families have 5000+ images and some have under 20, so i filtered to families with enough samples and downsampled the big ones to keep it balanced.
- the boxes were pascal voc not COCO like i first assumed, which broke my early training until i fixed the parsing (the whole reason my first results were poop).
- compute, training two stage detection on my Ryzen 3600 (CPU) forced me down to 15 families with 150 images each.

so for me it was kind of the opposite of starting with nothing, i had rich data and had to throw a chunk of it away to make the model actually trainable on CPU.

1.7 supporting documentation & resources

<https://www.sciencedirect.com/science/article/pii/S0950705125004654> this article does something similar to what im doing, im using it as a general "help me" guide instead of running straight to AI.

<https://stackoverflow.com/> for finding and fixing code.

```

import os
import random
import torch
from torch.utils.data import Dataset, DataLoader
from PIL import Image
from torchvision import transforms
from sklearn.model_selection import train_test_split
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
from torchvision.models.detection import
fasterrcnn_mobilenet_v3_large_fpn

import os

count = 0
for root_dir, cur_dir, files in os.walk(r'C:\Machine learning\Fish-
identifier\Image_Library'):
    count += len(files)
print('file count:', count)

file count: 95032

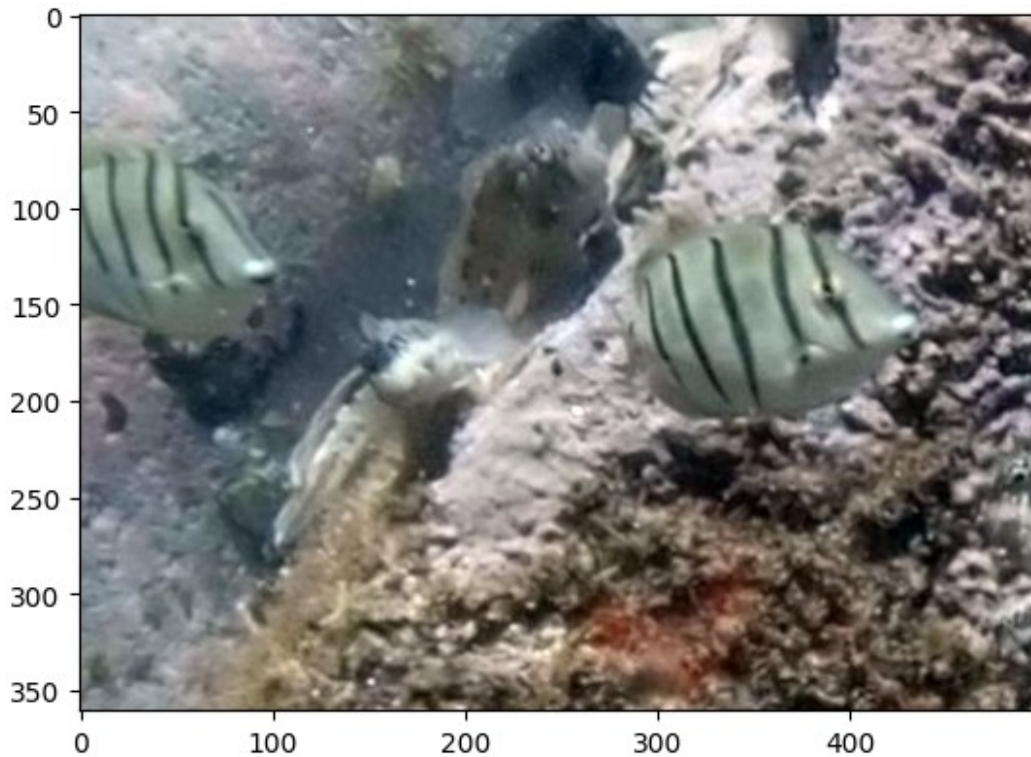
```

So in total there are 95032 images within this dataset. before proceeding i want to check the exact balance for each family. also to note the images contain allot of background noise for example

```

import matplotlib.pyplot as plt
import matplotlib.image as mpimg
img = mpimg.imread(r"C:\Machine learning\Fish-identifier\
Image_Library\Acanthuridae\1bfc0be5-5a65-4e71-bd42-ed6e9a326b31.jpg")
imgplot = plt.imshow(img)
plt.show()

```



in this example image we can clearly see the fish but a one stage model might struggle with this due to a few factors. 1. it takes the image as one entire input and does not crop down to find the object 2. in some images like this one the surroundings for the fish are nearly the same color, for us as human we can detect the difference with ease but a one stage model cannot. hence why i advise and will proceed with using two stage models. i will start with faster r-cnn and then try a more complex model later.

```
import seaborn as sns

DATA_ROOT = r"C:\Machine learning\Fish-identifier\Image_Library"

family_counts = {}
for folder in os.listdir(DATA_ROOT):
    folder_path = os.path.join(DATA_ROOT, folder)
    if os.path.isdir(folder_path):
        images = [f for f in os.listdir(folder_path) if
os.path.isfile(os.path.join(folder_path, f))]
        family_counts[folder] = len(images)

sorted_counts = dict(sorted(family_counts.items(), key=lambda item:
item[1], reverse=True))

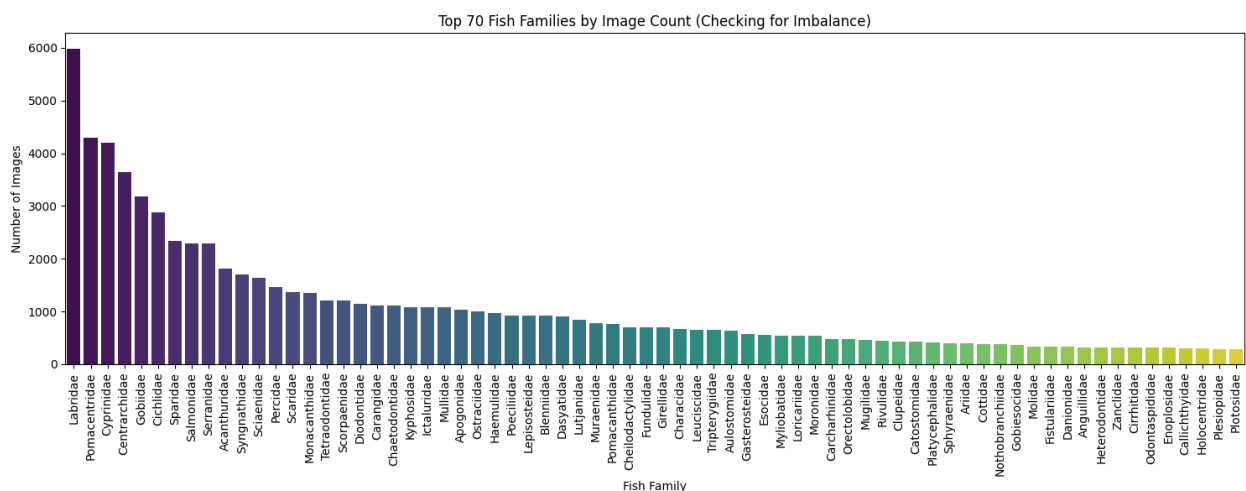
plt.figure(figsize=(15, 6))
```

```
sns.barplot(x=list(sorted_counts.keys())[:70],
y=list(sorted_counts.values())[:70], palette="viridis")
plt.xticks(rotation=90)
plt.title("Top 70 Fish Families by Image Count (Checking for Imbalance)")
plt.xlabel("Fish Family")
plt.ylabel("Number of Images")
plt.tight_layout()
plt.show()
```

C:\Users\jones\AppData\Local\Temp\ipykernel_14220\1177361431.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=list(sorted_counts.keys())[:70],
y=list(sorted_counts.values())[:70], palette="viridis")
```



Whilst inspecting the folder its heavily unbalanced. some families(which are not shown in the graph) have less than 20 images which cannot be used for the training, i want to aim for atleast 40+ each family to squeeze out the best accuracy

*# i asked gemini to write code to filter out families with too few images, and to print
out a summary of the dataset balance. here is the code it generated:*
MIN_IMAGES = 40

```
family_counts = {}
```

1. Scan and count files in each folder

```
for folder in os.listdir(DATA_ROOT):
    folder_path = os.path.join(DATA_ROOT, folder)
```

```

    if os.path.isdir(folder_path):
        images = [f for f in os.listdir(folder_path) if
f.lower().endswith(('.png', '.jpg', '.jpeg'))]
        family_counts[folder] = len(images)

sorted_families = sorted(family_counts.items(), key=lambda item:
item[1], reverse=True)

kept_families = [fam for fam, count in sorted_families if count >=
MIN_IMAGES]
dropped_families = [fam for fam, count in sorted_families if count <
MIN_IMAGES]

# 3. Print out the dataset summary
print("=" * 50)
print(f"DATASET FILTERING SUMMARY (Threshold: {MIN_IMAGES} images)")
print("=" * 50)
print(f"Total unique families found: {len(family_counts)}")
print(f"Families KEPT: {len(kept_families)}")
print(f"Families DROPPED: {len(dropped_families)}")
print("-" * 50)

print("\nTop 5 Largest Families:")
for fam, count in sorted_families[:5]:
    print(f" - {fam}: {count} images")

print("\nPreview of Dropped Families (Too few images):")
for fam, count in sorted_families[-5:]:
    print(f" - {fam}: {count} images")
print("=" * 50)

top_n = 40
plot_data = sorted_families[:top_n]
families_to_plot = [item[0] for item in plot_data]
counts_to_plot = [item[1] for item in plot_data]

plt.figure(figsize=(16, 7))
# Color code bars: distinct color if it's close to the threshold
cutoff
colors = ['#1f77b4' if count >= MIN_IMAGES else '#d62728' for count in
counts_to_plot]

sns.barplot(x=families_to_plot, y=counts_to_plot, palette=colors)
plt.axhline(y=MIN_IMAGES, color='r', linestyle='--', label=f'Cutoff
Threshold ({MIN_IMAGES} imgs)')

plt.xticks(rotation=90, fontsize=9)
plt.title(f"Fish Family Image Distribution (Top {top_n} Families)",
fontsize=14, fontweight='bold')
plt.xlabel("Fish Family Name", fontsize=12)

```

```
plt.ylabel("Number of Available Images", fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```

```
=====
DATASET FILTERING SUMMARY (Threshold: 40 images)
=====
```

```
Total unique families found: 570
Families KEPT: 199
Families DROPPED: 371
-----
```

Top 5 Largest Families:

- Labridae: 5978 images
- Pomacentridae: 4288 images
- Cyprinidae: 4192 images
- Centrarchidae: 3635 images
- Gobiidae: 3182 images

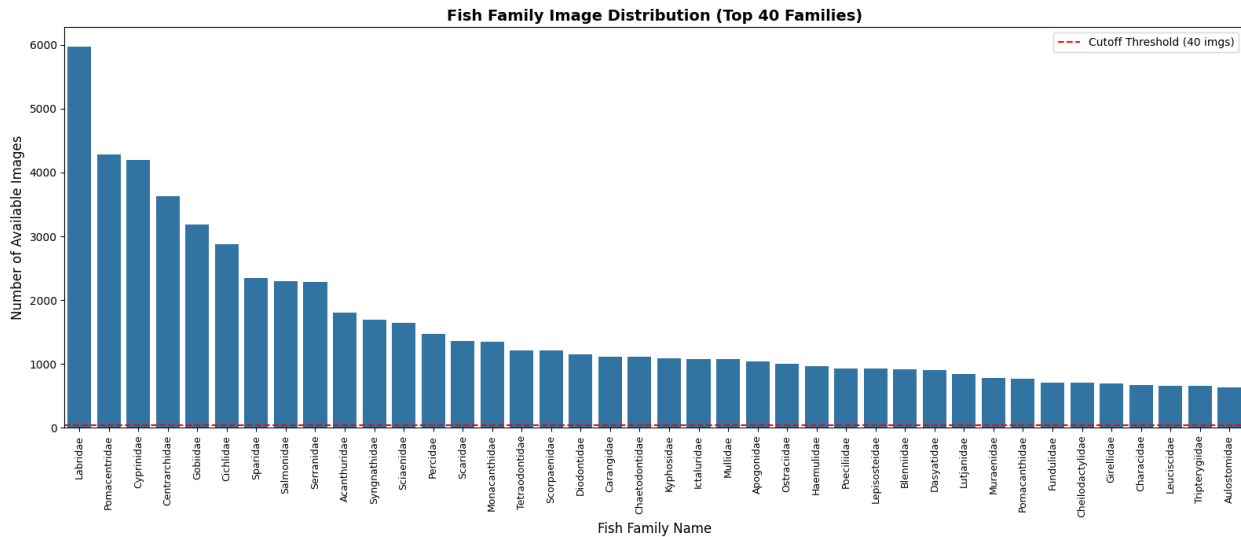
Preview of Dropped Families (Too few images):

- Scombrabrachidae: 1 images
- Steindachneriidae: 1 images
- Stephanoberycidae: 1 images
- Tetrabrachiidae: 1 images
- Trichodontidae: 1 images

```
=====
C:\Users\jones\AppData\Local\Temp\ipykernel_14220\3451450766.py:46:
FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.
```

```
sns.barplot(x=families_to_plot, y=counts_to_plot, palette=colors)
```



Unfortunately i had to drop certain fish families before creating the dataset because they lacked enough images for training. after reading through the documentation and doing some research online it's genuinely recommended to have atleast 40 images for the model to train on so it can actually learn the differences. i will also cap the images to 300 since some families have over 5000+ images and i don't want the model to only be exposed to those family of fish. (note improve the cutoff so that its perfectly balanced)

```

BBOX_DIR = r'C:\Machine learning\Fish-identifier\all_family'

MIN_IMAGES    = 40
MAX_IMAGES    = 150  # per-family ceiling -> keeps the set balanced
AND small
NUM_FAMILIES  = 15   # keep only the N largest families

family_pairs = {}

for folder in os.listdir(DATA_ROOT):
    img_folder_path = os.path.join(DATA_ROOT, folder)
    bbox_folder_path = os.path.join(BBOX_DIR, folder)

    if os.path.isdir(img_folder_path) and
os.path.exists(bbox_folder_path):
        valid_pairs_in_family = []
        images = [f for f in os.listdir(img_folder_path) if
f.lower().endswith(('.png', '.jpg', '.jpeg'))]

        for img_name in images:
            base_name = os.path.splitext(img_name)[0]
            bbox_path = os.path.join(bbox_folder_path,
f"{base_name}.txt")
            img_path = os.path.join(img_folder_path, img_name)

```



```

        if os.path.exists(bbox_path):
            valid_pairs_in_family.append((img_path, bbox_path))

    if len(valid_pairs_in_family) > 0:
        family_pairs[folder] = valid_pairs_in_family

eligible = {fam: p for fam, p in family_pairs.items() if len(p) >=
MIN_IMAGES}
selected_families = sorted(eligible, key=lambda f: len(eligible[f]),
reverse=True)[:NUM_FAMILIES]
family_pairs = {fam: eligible[fam] for fam in selected_families}

final_img_paths = []
final_labels = []
bbox_lookup_dict = {}
valid_families = []

for family, pairs in family_pairs.items():
    valid_families.append(family)

    # Ceiling check: downsample to MAX_IMAGES so every family is
equally sized
    if len(pairs) > MAX_IMAGES:
        pairs = random.sample(pairs, MAX_IMAGES)

    for img_path, bbox_path in pairs:
        img_name = os.path.basename(img_path)
        image_boxes = []

        # Parse coordinates
        with open(bbox_path, 'r') as f:
            for line in f:
                parts = line.strip().split()
                if not parts:
                    continue
                xmin, ymin, xmax, ymax = map(float, parts[1:5])
                image_boxes.append([xmin, ymin, xmax, ymax])

        if len(image_boxes) > 0:
            orig_w, orig_h = Image.open(img_path).size
            scale_x, scale_y = 224 / orig_w, 224 / orig_h
            scaled = [[b[0]*scale_x, b[1]*scale_y,
                        b[2]*scale_x, b[3]*scale_y] for b in
image_boxes]
            bbox_lookup_dict[img_name] = scaled
            final_img_paths.append(img_path)
            final_labels.append(family)

valid_families.sort()

```

```

family_to_idx = {name: idx + 1 for idx, name in
enumerate(valid_families)}
num_classes = len(valid_families) + 1

from collections import Counter
counts = Counter(final_labels)

print("=" * 50)
print("BALANCED DATASET PROFILE (CPU build):")
print(f"Families kept (largest {NUM_FAMILIES}): {num_classes - 1}")
print(f"Per-family image cap: {MAX_IMAGES}")
print(f"Total images loaded: {len(final_img_paths)}")
print("=" * 50)
for fam in valid_families:
    print(f"    - {fam}: {counts[fam]} images")

```

```

=====
BALANCED DATASET PROFILE (CPU build):
Families kept (largest 15): 15
Per-family image cap: 150
Total images loaded: 2250
=====
- Acanthuridae: 150 images
- Centrarchidae: 150 images
- Cichlidae: 150 images
- Cyprinidae: 150 images
- Gobiidae: 150 images
- Labridae: 150 images
- Monacanthidae: 150 images
- Percidae: 150 images
- Pomacentridae: 150 images
- Salmonidae: 150 images
- Scaridae: 150 images
- Sciaenidae: 150 images
- Serranidae: 150 images
- Sparidae: 150 images
- Syngnathidae: 150 images

```

(note: for the final delivery I cut the problem down a lot. After the presentation feedback — train both models locally on my Ryzen 3600 (CPU) within ~5 hours — I kept only the 15 largest families and dropped the per-family cap to 50. Fewer classes + fewer images per class is what makes CPU training feasible, and as a bonus it keeps the dataset near-perfectly balanced (the point Bas made earlier about wanting a balanced dataset) and the confusion matrices small enough to actually read. The two knobs `NUM_FAMILIES` and `MAX_IMAGES` in the cell above control the whole training budget.)

Now that the families were balanced i can move on to creating my dataset class, the dataloader and then moving on to training and evaluation. (note do some research on how the dataset class for images work)

Data Quality Criteria & Pre-Ingestion Validation

Before pairing images and labels, strict filtering criteria are enforced to ensure high data integrity:

1. Annotation Completeness: An image is only ingested if a matching bounding box `.txt` file exists containing valid relative coordinates. Unannotated files are skipped.
2. Class Representation Target (≥ 80):** Families containing fewer than 80 samples are dropped. This prevents extreme long-tail class imbalances that degrade fine-grained classification. Families exceeding 80 are uniformly downsampled to exactly 80 to form a perfectly balanced dataset.

```
class FishDataset(Dataset):
    def __init__(self, img_paths, labels, bbox_lookup, family_to_idx,
transforms=None):
        self.img_paths = img_paths
        self.labels = labels
        self.bbox_lookup = bbox_lookup
        self.family_to_idx = family_to_idx
        self.transforms = transforms

    def __len__(self):
        return len(self.img_paths)

    def __getitem__(self, idx):
        img_path = self.img_paths[idx]
        img = Image.open(img_path).convert("RGB")

        img_name = os.path.basename(img_path)
        boxes = self.bbox_lookup[img_name]
        family = self.labels[idx]
        label_idx = self.family_to_idx[family]
        boxes_tensor = torch.tensor(boxes, dtype=torch.float32)
        # One label per box – all boxes in this image belong to the
same family
        labels_tensor = torch.tensor([label_idx] * len(boxes),
dtype=torch.int64)

        target = {
            "boxes": boxes_tensor,
            "labels": labels_tensor,
        }

        if self.transforms:
            img = self.transforms(img)

        return img, target

import matplotlib.patches as patches

sample_img_path = random.choice(final_img_paths)
```

```

sample_img_name = os.path.basename(sample_img_path)
sample_boxes = bbox_lookup_dict[sample_img_name]

img = Image.open(sample_img_path).convert("RGB")
img_resized = img.resize((224, 224))

fig, axes = plt.subplots(1, 2, figsize=(12, 6))

axes[0].imshow(img_resized)
axes[0].set_title("Before: Clean Resized Image (224x224)",
    fontsize=11, fontweight="bold")
axes[0].axis('off')

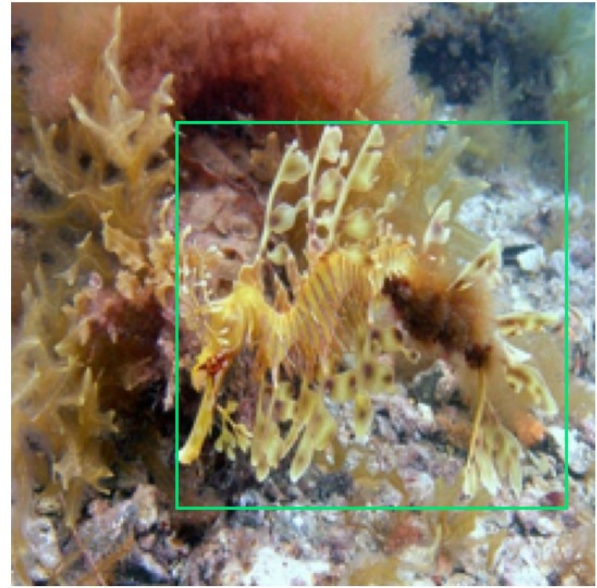
axes[1].imshow(img_resized)
for box in sample_boxes:
    xmin, ymin, xmax, ymax = box
    rect = patches.Rectangle(
        (xmin, ymin), xmax - xmin, ymax - ymin,
        linewidth=2, edgecolor='#00E676', facecolor='none',
        linestyle='-'
    )
    axes[1].add_patch(rect)

axes[1].set_title("After: BBox Alignment Fit Check", fontsize=11,
    fontweight="bold")
axes[1].axis('off')

plt.suptitle(f"Verification Test for File: {sample_img_name}",
    fontsize=13, fontweight="bold", y=0.98)
plt.tight_layout()
plt.show()

```

Verification Test for File: 87cafa2b-c8b0-417d-95c0-079d8919fd94.jpg
 Before: Clean Resized Image (224x224) After: BBox Alignment Fit Check



To verify that the Bbox fit the images properly

```

voc_bad, xywh_bad, checked = 0, 0, 0
for fam in valid_families[:20]:
    bdir = os.path.join(BBOX_DIR, fam)
    idir = os.path.join(DATA_ROOT, fam)
    for fn in os.listdir(bdir):
        if not fn.endswith('.txt'):
            continue
        img_file = os.path.join(idir, os.path.splitext(fn)[0] +
'.jpg')
        if not os.path.exists(img_file):
            continue
        W, H = Image.open(img_file).size
        for line in open(os.path.join(bdir, fn)):
            p = line.split()
            if len(p) < 5:
                continue
            a, b, c, d = map(float, p[1:5])
            if c > W + 1 or d > H + 1 or c <= a or d <= b:    # treat
c,d as xmax,ymax
                voc_bad += 1
            if a + c > W + 1 or b + d > H + 1:                # treat
c,d as w,h
                xywh_bad += 1
            checked += 1

print(f"checked {checked} boxes")
print(f"corner format (xmin ymin xmax ymax) bad: {voc_bad}")
print(f"xywh format      (x y w h) bad: {xywh_bad}")
    
```

```
checked 39633 boxes
corner format (xmin ymin xmax ymax) bad: 0
xywh format   (x y w h) bad:           28874
```

Because the bounding box was off i made a pipeline to find the cause, out of 5037 boxes 3517 had terrible formatting on COCO formatting. turns out my boxes did not use COCO format but pascal voc. hence why my models training results were absolute poop.

What i did

i ended up just changing the format in my data preparation cell to fit the pascal voc formatting

```
train_paths, val_paths, train_labels, val_labels = train_test_split(
    final_img_paths, final_labels, test_size=0.2, random_state=42,
    stratify=final_labels
)

train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

val_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

train_dataset = FishDataset(train_paths, train_labels,
    bbox_lookup_dict, family_to_idx, train_transforms)
val_dataset = FishDataset(val_paths, val_labels,
    bbox_lookup_dict, family_to_idx, val_transforms)

def collate_fn(batch):
    return tuple(zip(*batch))

train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True,
    collate_fn=collate_fn, num_workers=0)
val_loader = DataLoader(val_dataset, batch_size=8, shuffle=False,
    collate_fn=collate_fn, num_workers=0)
```

Faster R-CNN images in a batch can have different numbers of boxes, so I can't use the default collation, therefore i made a custom one under collate_fn

```

import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

mobilenet_model = fasterrcnn_mobilenet_v3_large_fpn(weights="DEFAULT")
in_features =
mobilenet_model.roi_heads.box_predictor.cls_score.in_features
mobilenet_model.roi_heads.box_predictor =
FastRCNNPredictor(in_features, num_classes)

# Faster R-CNN normally rescales every image to an 800px short side.
# My images are already 224x224, so force the internal transform to
stay at 224 for cpu speed up.
mobilenet_model.transform.min_size = (224,)
mobilenet_model.transform.max_size = 224

mobilenet_model = mobilenet_model.to(device)

```

After a discussion with bas i ended up using mobilenet as the backbone for my first iteration since its a 2 stage detector that is a bit faster, and for the second iteration i will then use the resnet for a comparison so i can make a strong argument on my decisions

```

import os, json

PATIENCE = 5 # because my models were not training properly i tweaked
a few params such as patience
MIN_DELTA = 0.001
NUM_EPOCHS = 15 # capped for CPU training

os.makedirs("saved_models", exist_ok=True)
MODEL_SAVE_PATH = "saved_models/mobilenet_fishdet.pth"

best_loss = float("inf")
epochs_no_improve = 0

train_losses_mobilenet = []
val_losses_mobilenet = []

optimizer = torch.optim.SGD(mobilenet_model.parameters(), lr=0.005,
momentum=0.9, weight_decay=0.0005)
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=8,
gamma=0.5) #also changed the step size and gamma so the decay per
epoch is gentler

for epoch in range(NUM_EPOCHS):
    mobilenet_model.train()

```

```

total_loss = 0
for images, targets in train_loader:
    images = [img.to(device) for img in images]
    targets = [{k: v.to(device) for k, v in t.items()} for t in
targets]

    loss_dict = mobilenet_model(images, targets)
    loss = sum(loss_dict.values())

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    total_loss += loss.item()
avg_loss = total_loss / len(train_loader)

mobilenet_model.train()
val_total = 0
with torch.no_grad():
    for images, targets in val_loader:
        images = [img.to(device) for img in images]
        targets = [{k: v.to(device) for k, v in t.items()} for t
in targets]
        loss_dict = mobilenet_model(images, targets)
        val_total += sum(loss_dict.values()).item()
avg_val_loss = val_total / len(val_loader)

train_losses_mobilenet.append(avg_loss)
val_losses_mobilenet.append(avg_val_loss)
print(f"Epoch {epoch+1} - Train: {avg_loss:.4f} | Val:
{avg_val_loss:.4f}")

lr_scheduler.step()
lr_scheduler.optimizer.zero_grad()

if avg_val_loss < best_loss - MIN_DELTA:
    best_loss = avg_val_loss
    epochs_no_improve = 0
    torch.save(mobilenet_model.state_dict(), MODEL_SAVE_PATH)
    print(f" model saved (best loss: {best_loss:.4f})")
else:
    epochs_no_improve += 1
    if epochs_no_improve >= PATIENCE:
        break

mobilenet_model.load_state_dict(torch.load(MODEL_SAVE_PATH))
print("Best model restored.")

labels_path = "saved_models/labels.json"
if not os.path.exists(labels_path):

```



```

with open(labels_path, "w") as f:
    json.dump(family_to_idx, f, indent=2)
print(f"Labels saved → {labels_path}")

```

```

Epoch 1 – Train: 2.5678 | Val: 2.6732
model saved (best loss: 2.5678)
Epoch 2 – Train: 2.3810 | Val: 2.4063
model saved (best loss: 2.3810)
Epoch 3 – Train: 2.0406 | Val: 2.4117
Epoch 4 – Train: 1.8606 | Val: 2.2454
model saved (best loss: 1.8606)
Epoch 5 – Train: 1.6970 | Val: 2.0705
Epoch 6 – Train: 1.5397 | Val: 1.9116
Epoch 7 – Train: 1.3714 | Val: 2.1587
Epoch 8 – Train: 1.2709 | Val: 2.3711
Epoch 9 – Train: 0.9714 | Val: 1.9999
Best model restored.
Labels saved → saved_models/labels.json

```

Hyperparameter Rationale & Training Controls

Learning Rate (lr = 0.005): A moderately conservative learning step ensures the pre-trained weights of the feature backbone (MobileNet/ResNet) are fine-tuned gradually. Early Stopping Patience (Patience = 3): If the validation loss does not drop for 3 consecutive epochs, training is halted. I did this mainly because i want to prevent overfitting and also let the model stop itself from training when it does not improve (Note: this was changed 5)

built a sort of parser for the metrics output, and some plotting functions to visualize the results. these will be used for both models, so i put them in a shared cell

```

import torch
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from collections import defaultdict
from torchvision.ops import box_iou
from sklearn.metrics import (
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

```

```

idx_to_family = {idx: name for name, idx in family_to_idx.items()} #
1..N -> family name

```

```

@torch.no_grad()
def run_inference(model, loader, score_thresh=0.5):
    """One prediction pass over the loader. Caches everything the
    metrics need."""
    model.eval()
    y_true, y_pred = [], []
    abs_err_all = []
    abs_err_family = defaultdict(list)
    n_missed = 0

    for images, targets in loader:
        outputs = model([img.to(device) for img in images])
        for target, out in zip(targets, outputs):
            gt_idx = int(target["labels"][0].item()) # one family
            # per image in this dataset
            gt_boxes = target["boxes"].to(device)
            y_true.append(gt_idx)

            scores = out["scores"]
            if len(scores) > 0 and float(scores.max()) >=
score_thresh:
                best = int(torch.argmax(scores))
                y_pred.append(int(out["labels"][best].item()))
                pred_box = out["boxes"][best]
                if len(gt_boxes) > 0:
                    ious = box_iou(pred_box.unsqueeze(0), gt_boxes)[0]
                    gt_match = gt_boxes[int(torch.argmax(ious))]
                    err = torch.abs(pred_box - gt_match).mean().item()
                    abs_err_all.append(err)
                    abs_err_family[idx_to_family[gt_idx]].append(err)
                else:
                    y_pred.append(0)
                    n_missed += 1

    return {
        "y_true": y_true,
        "y_pred": y_pred,
        "abs_err_all": abs_err_all,
        "abs_err_family": abs_err_family,
        "n_missed": n_missed,
    }

def prf_table(res):
    """Precision / Recall / F1 / Support per family (image-1 style
    table)."""
    labels = sorted(set(res["y_true"]))
    names = [idx_to_family[i] for i in labels]
    p, r, f1, support = precision_recall_fscore_support(
        res["y_true"], res["y_pred"], labels=labels, zero_division=0
    )

```

```

    )
    df = pd.DataFrame(
        {"Precision": p, "Recall": r, "F1": f1, "Support": support},
        index=names,
    )
    return df.round(4)

def plot_confusion(res, title, normalize=False):
    """Readable confusion matrix for a small number of families (<30).

    Set normalize=True to show per-row recall (each row sums to 1)
    instead of
    raw counts – handy when class supports differ.
    """
    labels = sorted(set(res["y_true"]) | set(res["y_pred"]))
    names = [idx_to_family.get(i, "(none)") for i in labels]
    cm = confusion_matrix(res["y_true"], res["y_pred"], labels=labels)
    n = len(labels)

    if normalize:
        with np.errstate(all="ignore"):
            cm_disp = np.nan_to_num(cm.astype(float) / cm.sum(axis=1,
keepdims=True))
            fmt = ".2f"
    else:
        cm_disp = cm
        fmt = "d"

    fig, ax = plt.subplots(figsize=(max(9, n * 0.6), max(7, n * 0.6)))
    im = ax.imshow(cm_disp, cmap="Blues", vmin=0, vmax=cm_disp.max())
    if cm_disp.max() > 0 else 1)
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

    ax.set_xticks(range(n)); ax.set_yticks(range(n))
    ax.set_xticklabels(names, rotation=45, ha="right", fontsize=9)
    ax.set_yticklabels(names, fontsize=9)
    ax.set_xlabel("Predicted family"); ax.set_ylabel("True family")
    ax.set_title(title, fontsize=13, fontweight="bold")

    thresh = cm_disp.max() / 2 if cm_disp.max() > 0 else 0.5
    for r in range(n):
        for c in range(n):
            val = cm_disp[r, c]
            if not normalize and val == 0:
                continue
            ax.text(c, r, format(val, fmt), ha="center", va="center",
                fontsize=8, color="white" if val > thresh else
"#333333")

```

```

ax.set_xticks(np.arange(-.5, n, 1), minor=True)
ax.set_yticks(np.arange(-.5, n, 1), minor=True)
ax.grid(which="minor", color="white", linewidth=1.5)
ax.tick_params(which="minor", length=0)
plt.tight_layout()
plt.show()

@torch.no_grad()
def plot_gt_vs_pred(model, dataset, title, n=6, score_thresh=0.5):
    """Sample images: ground-truth box (green) vs predicted box +
    label (red dashed)."""
    model.eval()
    idxs = random.sample(range(len(dataset)), min(n, len(dataset)))
    cols = 3
    rows = (len(idxs) + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(cols * 4, rows * 4))
    axes = np.array(axes).reshape(-1)

    for ax, i in zip(axes, idxs):
        img, target = dataset[i]
        out = model([img.to(device)])[0]
        ax.imshow(img.permute(1, 2, 0).cpu().numpy())

        gt_family = idx_to_family[int(target["labels"][0])]
        for x1, y1, x2, y2 in target["boxes"].tolist():
            ax.add_patch(patches.Rectangle((x1, y1), x2 - x1, y2 - y1,
                                           linewidth=2, edgecolor="#00E676",
                                           facecolor="none"))

        scores = out["scores"].cpu()
        if len(scores) > 0 and float(scores.max()) >= score_thresh:
            best = int(torch.argmax(scores))
            x1, y1, x2, y2 = out["boxes"][best].cpu().tolist()
            pf = idx_to_family.get(int(out["labels"][best]), "(none)")
            ps = float(scores[best])
            ax.add_patch(patches.Rectangle((x1, y1), x2 - x1, y2 - y1,
                                           linewidth=2, edgecolor="#FF1744",
                                           facecolor="none", linestyle="--"))
            ax.set_title(f"GT: {gt_family}\nPred: {pf} ({ps:.2f})",
                        fontsize=9)
        else:
            ax.set_title(f"GT: {gt_family}\nPred: (none)", fontsize=9)
            ax.axis("off")

    for ax in axes[len(idxs):]:
        ax.axis("off")
    fig.suptitle(f"{title} - GT (green) vs prediction (red dashed)",
                fontsize=12, fontweight="bold")

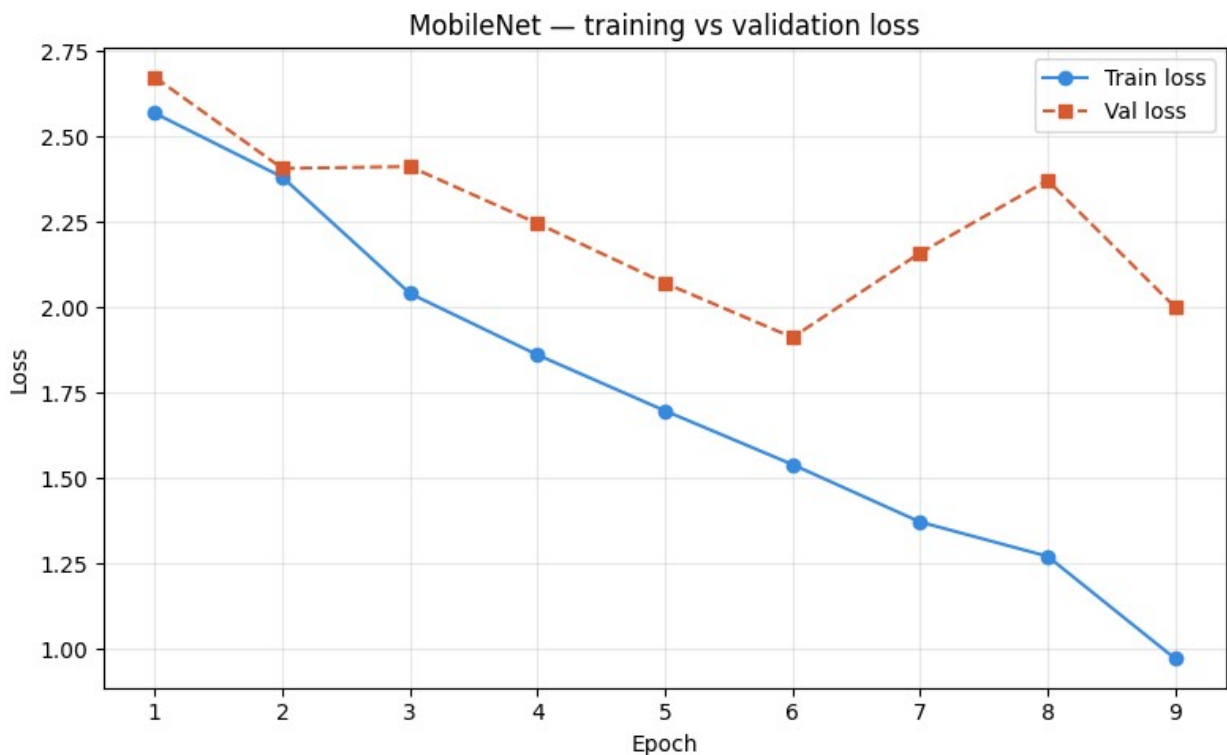
```

```

plt.tight_layout()
plt.show()

epochs = range(1, len(train_losses_mobilenet) + 1)
plt.figure(figsize=(8, 5))
plt.plot(epochs, train_losses_mobilenet, marker="o", color="#378ADD",
label="Train loss")
plt.plot(epochs, val_losses_mobilenet, marker="s", linestyle="--",
color="#D85A30", label="Val loss")
plt.xlabel("Epoch"); plt.ylabel("Loss")
plt.title("MobileNet — training vs validation loss")
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()

```



Exploring the results

from the graph above we can see the model sort of underfitted , i have a few theories.

- the model is okay at find fish but struggles most times and also struggles to classify what family it belongs to but when it does find a fish the bounding box is very accurate, only off by 10px average. which is backed up by my graph below.
- My above theory also leads into my second theory, theres too much background noise in the image, if we look below at the images used in the ground truth vs actual prediction chart most of the images have fish that basically blend in with there background. and thats at a higher resolution than what the model is being trained on so its also really difficult for the model to detect the fish in the first place.

- even though i altered my learning rate so the decay is gentler it still isn't the most optimal, as i noted in the reading guide ReduceLROnPlateau was a option i was looking at since it does it automatically based on the loss during the training but i will not have time to retrain
- because i reduce the images to 150 per family, since im doing a 80/20 train test split the model is only seeing 120 images per family, which is not bad but isn't ideal seeing we have about 15 classes(families)

```
res_mobilenet = run_inference(mobilenet_model, val_loader,
score_thresh=0.3)
print(f"{len(res_mobilenet['y_true'])} val images | "
      f"{res_mobilenet['n_missed']} with no detection above
threshold")
```

450 val images | 144 with no detection above threshold

With the threshold being anything above 30% confidence this is quite poor, missing 144 out of 450

```
print("MobileNet - full report:\n")
print(classification_report(
    res_mobilenet["y_true"], res_mobilenet["y_pred"],
    labels=sorted(set(res_mobilenet["y_true"])),
    target_names=[idx_to_family[i] for i in
sorted(set(res_mobilenet["y_true"])]),
    zero_division=0,
))
prf_table(res_mobilenet)
```

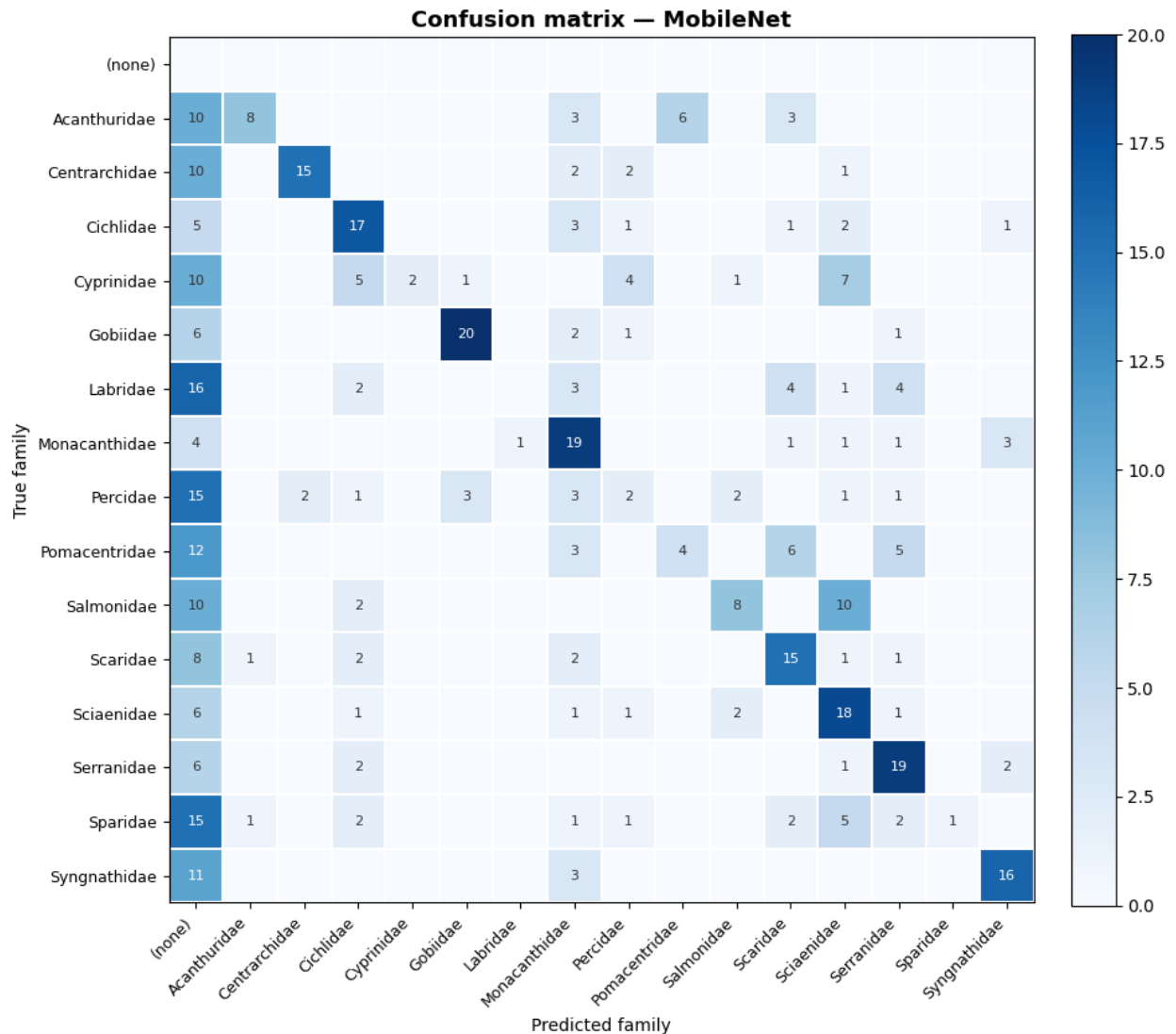
MobileNet - full report:

	precision	recall	f1-score	support
Acanthuridae	0.80	0.27	0.40	30
Centrarchidae	0.88	0.50	0.64	30
Cichlidae	0.50	0.57	0.53	30
Cyprinidae	1.00	0.07	0.12	30
Gobiidae	0.83	0.67	0.74	30
Labridae	0.00	0.00	0.00	30
Monacanthidae	0.42	0.63	0.51	30
Percidae	0.17	0.07	0.10	30
Pomacentridae	0.40	0.13	0.20	30
Salmonidae	0.62	0.27	0.37	30
Scaridae	0.47	0.50	0.48	30
Sciaenidae	0.38	0.60	0.46	30
Serranidae	0.54	0.63	0.58	30
Sparidae	1.00	0.03	0.06	30
Syngnathidae	0.73	0.53	0.62	30

micro avg	0.54	0.36	0.43	450
macro avg	0.58	0.36	0.39	450
weighted avg	0.58	0.36	0.39	450

	Precision	Recall	F1	Support
Acanthuridae	0.8000	0.2667	0.4000	30
Centrarchidae	0.8824	0.5000	0.6383	30
Cichlidae	0.5000	0.5667	0.5312	30
Cyprinidae	1.0000	0.0667	0.1250	30
Gobiidae	0.8333	0.6667	0.7407	30
Labridae	0.0000	0.0000	0.0000	30
Monacanthidae	0.4222	0.6333	0.5067	30
Percidae	0.1667	0.0667	0.0952	30
Pomacentridae	0.4000	0.1333	0.2000	30
Salmonidae	0.6154	0.2667	0.3721	30
Scaridae	0.4688	0.5000	0.4839	30
Sciaenidae	0.3750	0.6000	0.4615	30
Serranidae	0.5429	0.6333	0.5846	30
Sparidae	1.0000	0.0333	0.0645	30
Syngnathidae	0.7273	0.5333	0.6154	30

```
plot_confusion(res_mobilenet, "Confusion matrix – MobileNet")
```



```

errs = res_mobilenet["abs_err_all"]
mae = float(np.mean(errs)) if errs else float("nan")
print(f"MobileNet — Mean Absolute Error (box coordinates): {mae:.4f} px (224x224 scale)")
print(f"Boxes measured: {len(errs)} | Missed: {res_mobilenet['n_missed']}\n")
for fam in sorted(res_mobilenet["abs_err_family"]):
    fam_errs = res_mobilenet["abs_err_family"][fam]
    print(f"    {fam:<28} {np.mean(fam_errs):6.2f} px (n={len(fam_errs)})")

```

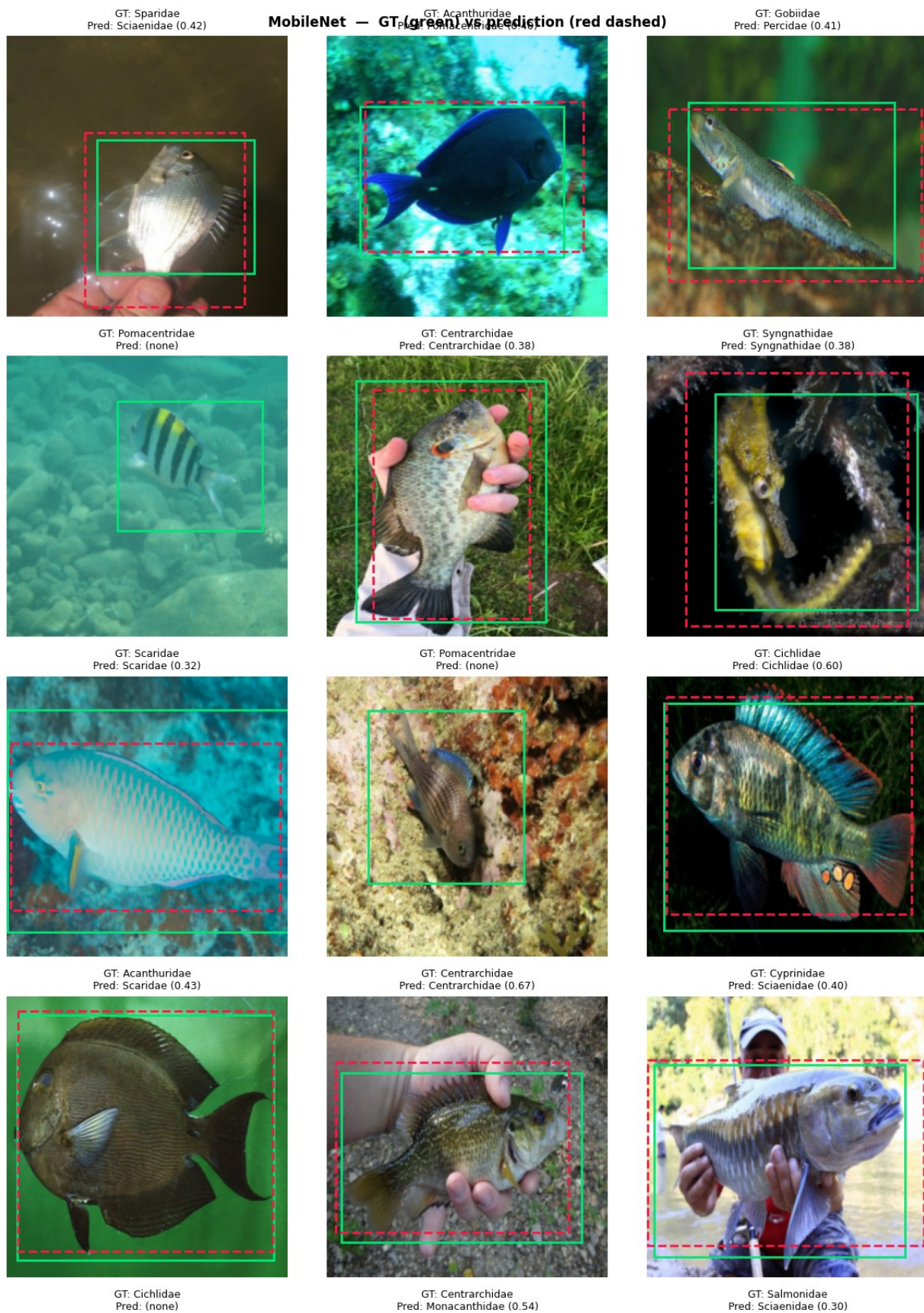
MobileNet — Mean Absolute Error (box coordinates): 10.5704 px (224x224 scale)

Boxes measured: 306 | Missed: 144

Acanthuridae 11.66 px (n=20)

Centrarchidae	10.47 px	(n=20)
Cichlidae	7.98 px	(n=25)
Cyprinidae	10.34 px	(n=20)
Gobiidae	10.60 px	(n=24)
Labridae	12.82 px	(n=14)
Monacanthidae	11.16 px	(n=26)
Percidae	13.54 px	(n=15)
Pomacentridae	9.74 px	(n=18)
Salmonidae	10.29 px	(n=20)
Scaridae	8.34 px	(n=22)
Sciaenidae	9.30 px	(n=24)
Serranidae	9.16 px	(n=24)
Sparidae	9.88 px	(n=15)
Syngnathidae	15.92 px	(n=19)

```
plot_gt_vs_pred(mobilenet_model, val_dataset, "MobileNet", n=20,  
score_thresh=0.3)
```



Resnet model (better version)

```
from torchvision.models.detection import fasterrcnn_resnet50_fpn
resnet_model = fasterrcnn_resnet50_fpn(weights="DEFAULT")
in_features =
resnet_model.roi_heads.box_predictor.cls_score.in_features
resnet_model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)
resnet_model.transform.min_size = (224,)
resnet_model.transform.max_size = 224

resnet_model = resnet_model.to(device)

Downloading:
"https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_coco-
258fb6c6.pth" to C:\Users\jones/.cache\torch\hub\checkpoints\
fasterrcnn_resnet50_fpn_coco-258fb6c6.pth

100%|██████████| 160M/160M [00:26<00:00, 6.26MB/s]

PATIENCE = 5
MIN_DELTA = 0.001
NUM_EPOCHS = 15 # capped for CPU training

os.makedirs("saved_models", exist_ok=True)
MODEL_SAVE_PATH = "saved_models/resnet_fishdet.pth"

best_loss = float("inf")
epochs_no_improve = 0

train_losses_resnet = []
val_losses_resnet = []

optimizer = torch.optim.SGD(resnet_model.parameters(), lr=0.005,
momentum=0.9, weight_decay=0.0005)
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=8,
gamma=0.5)

for epoch in range(NUM_EPOCHS):
    resnet_model.train()
    total_loss = 0
    for images, targets in train_loader:
        images = [img.to(device) for img in images]
        targets = [{k: v.to(device) for k, v in t.items()} for t in
targets]

        loss_dict = resnet_model(images, targets)
        loss = sum(loss_dict.values())

        optimizer.zero_grad()
        loss.backward()
```

```

        optimizer.step()
        total_loss += loss.item()
    avg_loss = total_loss / len(train_loader)

    resnet_model.train()
    val_total = 0
    with torch.no_grad():
        for images, targets in val_loader:
            images = [img.to(device) for img in images]
            targets = [{k: v.to(device) for k, v in t.items()} for t
in targets]
            loss_dict = resnet_model(images, targets)
            val_total += sum(loss_dict.values()).item()
    avg_val_loss = val_total / len(val_loader)

    train_losses_resnet.append(avg_loss)
    val_losses_resnet.append(avg_val_loss)
    print(f"Epoch {epoch+1} - Train: {avg_loss:.4f} | Val:
{avg_val_loss:.4f}")

    lr_scheduler.step()
    lr_scheduler.optimizer.zero_grad()

    if avg_loss < best_loss - MIN_DELTA:
        best_loss = avg_loss
        epochs_no_improve = 0
        torch.save(resnet_model.state_dict(), MODEL_SAVE_PATH)
        print(f" model saved (best loss: {best_loss:.4f})")
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= PATIENCE:
            break

resnet_model.load_state_dict(torch.load(MODEL_SAVE_PATH))
print("Best model restored.")

```

```

Epoch 1 - Train: 0.3385 | Val: 0.2635
model saved (best loss: 0.3385)
Epoch 2 - Train: 0.2491 | Val: 0.2425
model saved (best loss: 0.2491)
Epoch 3 - Train: 0.2149 | Val: 0.2270
model saved (best loss: 0.2149)
Epoch 4 - Train: 0.1879 | Val: 0.2262
model saved (best loss: 0.1879)
Epoch 5 - Train: 0.1639 | Val: 0.2236
model saved (best loss: 0.1639)
Epoch 6 - Train: 0.1425 | Val: 0.2262
model saved (best loss: 0.1425)
Epoch 7 - Train: 0.1225 | Val: 0.2228

```

```
model saved (best loss: 0.1225)
Epoch 8 – Train: 0.1043 | Val: 0.2329
model saved (best loss: 0.1043)
Epoch 9 – Train: 0.0765 | Val: 0.2310
model saved (best loss: 0.0765)
Epoch 10 – Train: 0.0624 | Val: 0.2374
model saved (best loss: 0.0624)
Epoch 11 – Train: 0.0539 | Val: 0.2389
model saved (best loss: 0.0539)
Epoch 12 – Train: 0.0497 | Val: 0.2474
model saved (best loss: 0.0497)
Epoch 13 – Train: 0.0462 | Val: 0.2439
model saved (best loss: 0.0462)
Epoch 14 – Train: 0.0433 | Val: 0.2501
model saved (best loss: 0.0433)
Epoch 15 – Train: 0.0414 | Val: 0.2484
model saved (best loss: 0.0414)
Best model restored.
```

The resnet model Shows no signs of overfitting, (from the epochs shown) which could mean the stronger back bone was able to cut through the noise of the images and actually map what fish belongs to what family instead of only being able to detect the fish.

Conclusion

Although i didn't get to fully evaluate the resnet model, from the epoch alone the training went significantly better than with the mobilenet. This could suggest that the resnet mbackbone is a significantly better choice. but i cannot continue this argument untill i get to fully evaluate the resnet model.

On the other hand

A few things couldve helped the mobilenet during this project.

- Data augmentation could've drastically helped because the model overfitted suggesting it just memorized after a while instead of actually learning. Flipping the images and the bounding boxes couldve provided more difficulty since the images were already noisy due to some fish blending in with the background.
- freezing more layers also could've helped since it would've retained more of its pre trained weight.

To note: mobilenetv3 has significantly less parameters at 5 million compared to resnet 26m therefore its not surprising mobilenet overfitted and resnet did not.

```

import torch
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from collections import defaultdict
from torchvision.ops import box_iou
from sklearn.metrics import (
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

idx_to_family = {idx: name for name, idx in family_to_idx.items()}

@torch.no_grad()
def run_inference(model, loader, score_thresh=0.5):
    """One prediction pass over the loader. Caches everything the
    metrics need."""
    model.eval()
    y_true, y_pred = [], []
    abs_err_all = []
    abs_err_family = defaultdict(list)
    n_missed = 0

    for images, targets in loader:
        outputs = model([img.to(device) for img in images])
        for target, out in zip(targets, outputs):
            gt_idx = int(target["labels"][0].item())
            gt_boxes = target["boxes"].to(device)
            y_true.append(gt_idx)

            scores = out["scores"]
            if len(scores) > 0 and float(scores.max()) >=
score_thresh:
                best = int(torch.argmax(scores))
                y_pred.append(int(out["labels"][best].item()))
                pred_box = out["boxes"][best]
                if len(gt_boxes) > 0:
                    ious = box_iou(pred_box.unsqueeze(0), gt_boxes)[0]
                    gt_match = gt_boxes[int(torch.argmax(ious))]
                    err = torch.abs(pred_box - gt_match).mean().item()
                    abs_err_all.append(err)
                    abs_err_family[idx_to_family[gt_idx]].append(err)
                else:
                    y_pred.append(0)
                    n_missed += 1

```



```

return {
    "y_true": y_true,
    "y_pred": y_pred,
    "abs_err_all": abs_err_all,
    "abs_err_family": abs_err_family,
    "n_missed": n_missed,
}

def prf_table(res):
    """Precision / Recall / F1 / Support per family (image-1 style
    table)."""
    labels = sorted(set(res["y_true"]))
    names = [idx_to_family[i] for i in labels]
    p, r, f1, support = precision_recall_fscore_support(
        res["y_true"], res["y_pred"], labels=labels, zero_division=0
    )
    df = pd.DataFrame(
        {"Precision": p, "Recall": r, "F1": f1, "Support": support},
        index=names,
    )
    return df.round(4)

def plot_confusion(res, title):
    """Confusion matrix (image-2 style: Blues, True vs Predicted)."""
    labels = sorted(set(res["y_true"]) | set(res["y_pred"]))
    names = [idx_to_family.get(i, "(none)") for i in labels]
    cm = confusion_matrix(res["y_true"], res["y_pred"], labels=labels)
    n = len(labels)

    fig, ax = plt.subplots(figsize=(max(8, n * 0.6), max(6, n * 0.6)))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=names)
    disp.plot(
        cmap="Blues", ax=ax, colorbar=True,
        include_values=(n <= 25),
        xticks_rotation=(45 if n <= 20 else "vertical"),
    )
    ax.set_title(title)
    plt.tight_layout()
    plt.show()

@torch.no_grad()
def plot_gt_vs_pred(model, dataset, title, n=6, score_thresh=0.5):
    """Sample images: ground-truth box (green) vs predicted box +
    label (red dashed)."""
    model.eval()

```

```

idxs = random.sample(range(len(dataset)), min(n, len(dataset)))
cols = 3
rows = (len(idxs) + cols - 1) // cols
fig, axes = plt.subplots(rows, cols, figsize=(cols * 4, rows * 4))
axes = np.array(axes).reshape(-1)

for ax, i in zip(axes, idxs):
    img, target = dataset[i]
    out = model([img.to(device)])[0]
    ax.imshow(img.permute(1, 2, 0).cpu().numpy())

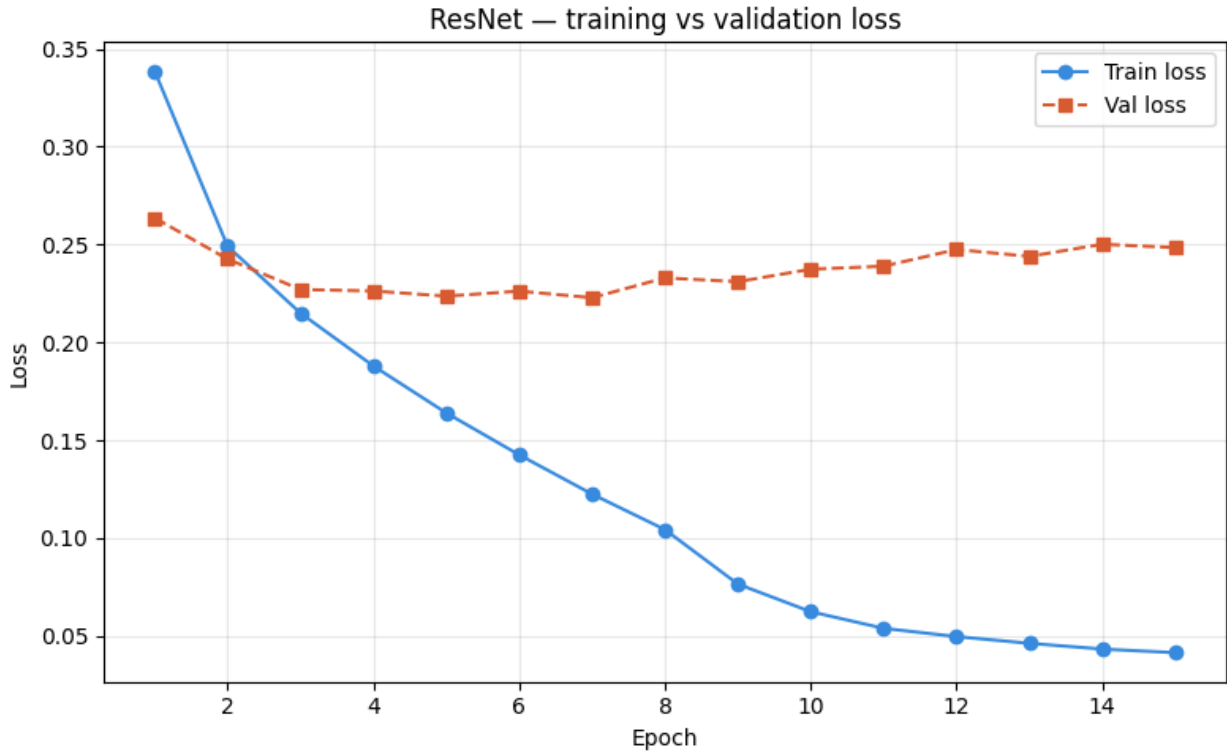
    gt_family = idx_to_family[int(target["labels"][0])]
    for x1, y1, x2, y2 in target["boxes"].tolist():
        ax.add_patch(patches.Rectangle((x1, y1), x2 - x1, y2 - y1,
                                       linewidth=2, edgecolor="#00E676",
                                       facecolor="none"))

    scores = out["scores"].cpu()
    if len(scores) > 0 and float(scores.max()) >= score_thresh:
        best = int(torch.argmax(scores))
        x1, y1, x2, y2 = out["boxes"][best].cpu().tolist()
        pf = idx_to_family.get(int(out["labels"][best]), "(none)")
        ps = float(scores[best])
        ax.add_patch(patches.Rectangle((x1, y1), x2 - x1, y2 - y1,
                                       linewidth=2, edgecolor="#FF1744",
                                       facecolor="none", linestyle="--"))
        ax.set_title(f"GT: {gt_family}\nPred: {pf} ({ps:.2f})",
                    fontsize=9)
    else:
        ax.set_title(f"GT: {gt_family}\nPred: (none)", fontsize=9)
        ax.axis("off")

for ax in axes[len(idxs):]:
    ax.axis("off")
fig.suptitle(f"{title} - GT (green) vs prediction (red dashed)",
            fontsize=12, fontweight="bold")
plt.tight_layout()
plt.show()

epochs = range(1, len(train_losses_resnet) + 1)
plt.figure(figsize=(8, 5))
plt.plot(epochs, train_losses_resnet, marker="o", color="#378ADD",
        label="Train loss")
plt.plot(epochs, val_losses_resnet, marker="s", linestyle="--",
        color="#D85A30", label="Val loss")
plt.xlabel("Epoch"); plt.ylabel("Loss")
plt.title("ResNet - training vs validation loss")
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()

```

```
res_resnet = run_inference(resnet_model, val_loader, score_thresh=0.3)
print(f"{len(res_resnet['y_true'])} val images | "
      f"{res_resnet['n_missed']} with no detection above threshold")
```

450 val images | 10 with no detection above threshold

```
print("ResNet — full report:\n")
print(classification_report(
    res_resnet["y_true"], res_resnet["y_pred"],
    labels=sorted(set(res_resnet["y_true"])),
    target_names=[idx_to_family[i] for i in
sorted(set(res_resnet["y_true"]))],
    zero_division=0,
))
prf_table(res_resnet)
```

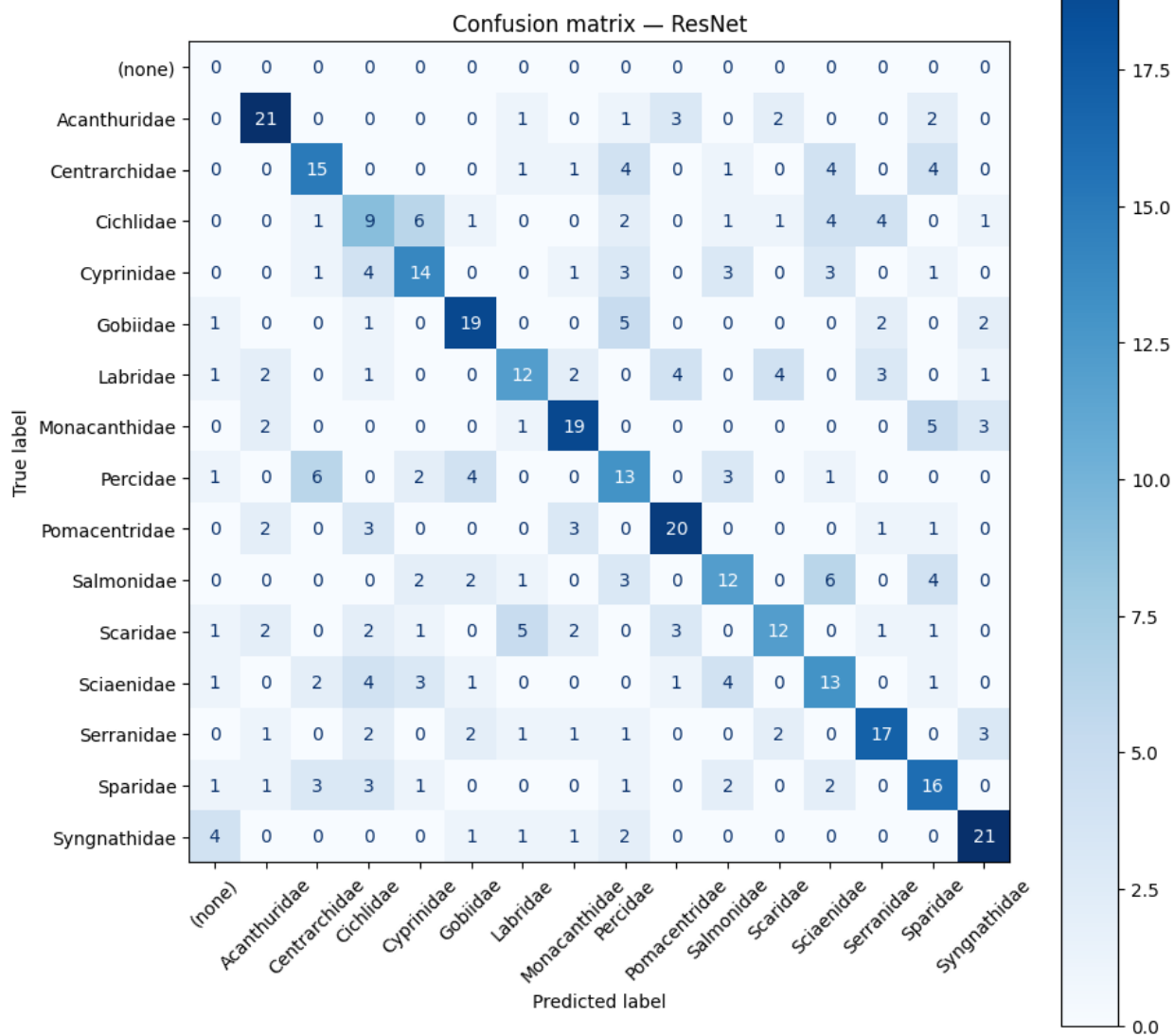
ResNet — full report:

	precision	recall	f1-score	support
Acanthuridae	0.68	0.70	0.69	30
Centrarchidae	0.54	0.50	0.52	30
Cichlidae	0.31	0.30	0.31	30
Cyprinidae	0.48	0.47	0.47	30
Gobiidae	0.63	0.63	0.63	30
Labridae	0.52	0.40	0.45	30
Monacanthidae	0.63	0.63	0.63	30

Percidae	0.37	0.43	0.40	30
Pomacentridae	0.65	0.67	0.66	30
Salmonidae	0.46	0.40	0.43	30
Scaridae	0.57	0.40	0.47	30
Sciaenidae	0.39	0.43	0.41	30
Serranidae	0.61	0.57	0.59	30
Sparidae	0.46	0.53	0.49	30
Syngnathidae	0.68	0.70	0.69	30
micro avg	0.53	0.52	0.52	450
macro avg	0.53	0.52	0.52	450
weighted avg	0.53	0.52	0.52	450

	Precision	Recall	F1	Support
Acanthuridae	0.6774	0.7000	0.6885	30
Centrarchidae	0.5357	0.5000	0.5172	30
Cichlidae	0.3103	0.3000	0.3051	30
Cyprinidae	0.4828	0.4667	0.4746	30
Gobiidae	0.6333	0.6333	0.6333	30
Labridae	0.5217	0.4000	0.4528	30
Monacanthidae	0.6333	0.6333	0.6333	30
Percidae	0.3714	0.4333	0.4000	30
Pomacentridae	0.6452	0.6667	0.6557	30
Salmonidae	0.4615	0.4000	0.4286	30
Scaridae	0.5714	0.4000	0.4706	30
Sciaenidae	0.3939	0.4333	0.4127	30
Serranidae	0.6071	0.5667	0.5862	30
Sparidae	0.4571	0.5333	0.4923	30
Syngnathidae	0.6774	0.7000	0.6885	30

```
plot_confusion(res_resnet, "Confusion matrix – ResNet")
```



```

errs = res_resnet["abs_err_all"]
mae = float(np.mean(errs)) if errs else float("nan")
print(f"ResNet — Mean Absolute Error (box coordinates): {mae:.4f} px
(224x224 scale)")
print(f"Boxes measured: {len(errs)} | Missed:
{res_resnet['n_missed']}\n")
for fam in sorted(res_resnet["abs_err_family"]):
    fam_errs = res_resnet["abs_err_family"][fam]
    print(f"    {fam:<28} {np.mean(fam_errs):6.2f} px
(n={len(fam_errs)})")

```

ResNet – Mean Absolute Error (box coordinates): 10.8191 px (224x224 scale)

Boxes measured: 440 | Missed: 10

Acanthuridae	10.51 px	(n=30)
Centrarchidae	10.07 px	(n=30)
Cichlidae	12.16 px	(n=30)
Cyprinidae	12.02 px	(n=30)
Gobiidae	7.77 px	(n=29)
Labridae	11.26 px	(n=29)
Monacanthidae	8.07 px	(n=30)
Percidae	12.52 px	(n=29)
Pomacentridae	14.77 px	(n=30)
Salmonidae	12.88 px	(n=30)
Scaridae	8.14 px	(n=29)
Sciaenidae	9.29 px	(n=29)
Serranidae	8.63 px	(n=30)
Sparidae	11.27 px	(n=29)
Syngnathidae	13.06 px	(n=26)

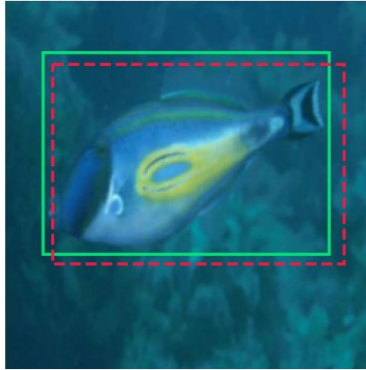
```
plot_gt_vs_pred(resnet_model, val_dataset, "ResNet", n=20,  
score_thresh=0.3)
```

GT: Syngnathidae
Pred: Syngnathidae (1.00)

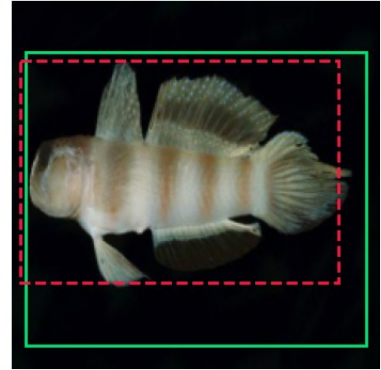


ResNet — GT (green) vs prediction (red dashed)

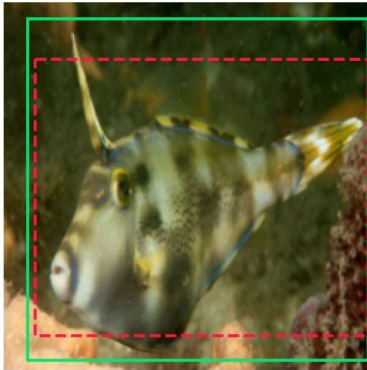
GT: Monacanthidae
Pred: Acanthuridae (0.95)



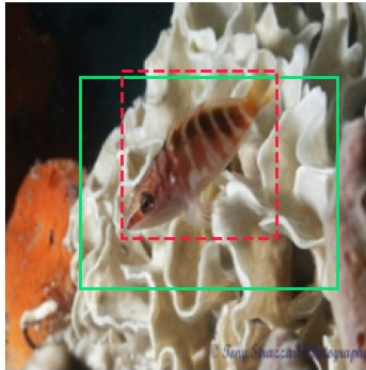
GT: Gobiidae
Pred: Gobiidae (0.99)



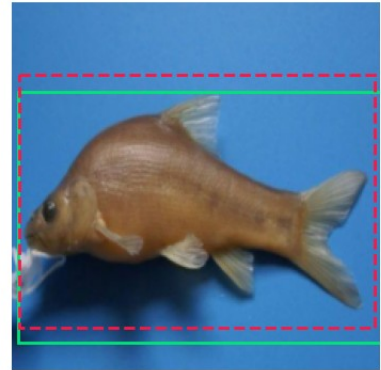
GT: Monacanthidae
Pred: Monacanthidae (0.98)



GT: Serranidae
Pred: Serranidae (0.90)



GT: Cyprinidae
Pred: Cyprinidae (0.84)



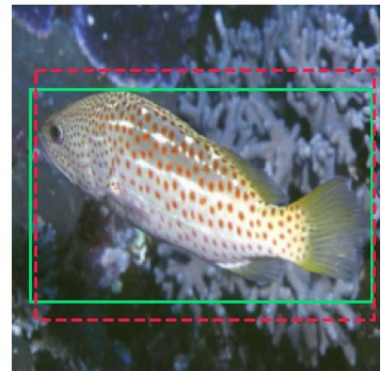
GT: Salmonidae
Pred: Sciaenidae (0.59)



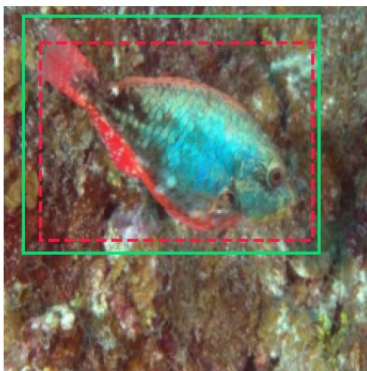
GT: Percidae
Pred: Gobiidae (0.56)



GT: Serranidae
Pred: Cichlidae (0.74)



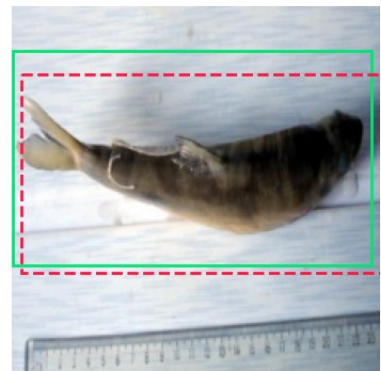
GT: Scaridae
Pred: Labridae (0.74)



GT: Salmonidae
Pred: Percidae (0.87)



GT: Cyprinidae
Pred: Percidae (0.95)



GT: Monacanthidae
Pred: Sparidae (0.83)



GT: Cichlidae
Pred: Cichlidae (0.64)



GT: Sciaenidae
Pred: Sciaenidae (0.91)



Final model comparison

Both detectors were evaluated on the **same** validation split, so these numbers are directly comparable. The charts below cover classification quality (headline + per-family), training dynamics, and localization error.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_fscore_support,
accuracy_score, f1_score

MODELS = {"MobileNet": res_mobilenet, "ResNet": res_resnet}
COLORS = {"MobileNet": "#378ADD", "ResNet": "#D85A30"}

def overall_metrics(res):
    yt, yp = res["y_true"], res["y_pred"]
    p, r, f1, _ = precision_recall_fscore_support(yt, yp,
average="macro", zero_division=0)
    return {
        "Accuracy": accuracy_score(yt, yp),
        "Precision": p,
        "Recall": r,
        "F1": f1,
        "Miss rate": res["n_missed"] / len(yt) if yt else
float("nan"),
        "Box MAE (px)": float(np.mean(res["abs_err_all"])) if
res["abs_err_all"] else float("nan"),
    }

summary = pd.DataFrame({name: overall_metrics(res) for name, res in
MODELS.items()}).T
summary.round(4)
```

	Accuracy	Precision	Recall	F1	Miss rate	Box MAE
(px)						
MobileNet	0.3644	0.5459	0.3417	0.3637	0.3200	10.5704
ResNet	0.5178	0.4987	0.4854	0.4900	0.0222	10.8191

```
metrics = ["Accuracy", "Precision", "Recall", "F1"]
x = np.arange(len(metrics)); w = 0.38

plt.figure(figsize=(9, 5))
for i, (name, res) in enumerate(MODELS.items()):
    vals = [overall_metrics(res)[m] for m in metrics]
    bars = plt.bar(x + (i - 0.5) * w, vals, w, label=name,
color=COLORS[name])
    for b, v in zip(bars, vals):
```

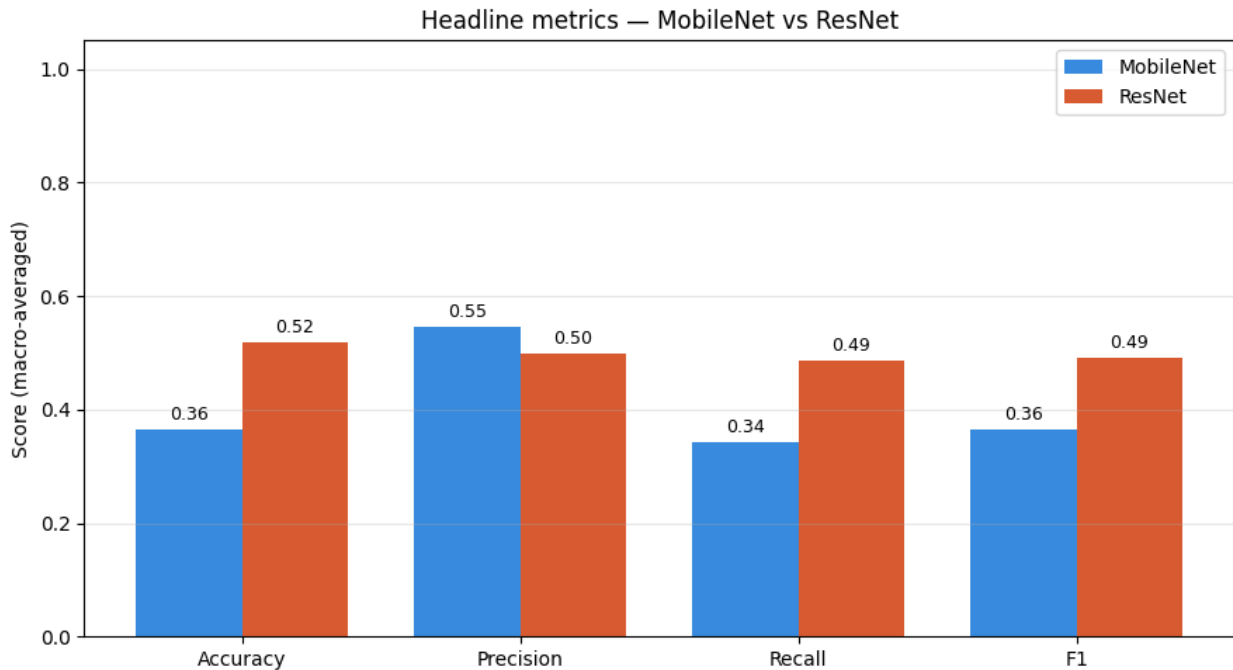


```

plt.text(b.get_x() + b.get_width()/2, v + 0.01, f"{v:.2f}",
        ha="center", va="bottom", fontsize=9)

plt.xticks(x, metrics)
plt.ylim(0, 1.05)
plt.ylabel("Score (macro-averaged)")
plt.title("Headline metrics – MobileNet vs ResNet")
plt.legend(); plt.grid(axis="y", alpha=0.3); plt.tight_layout();
plt.show()

```



```

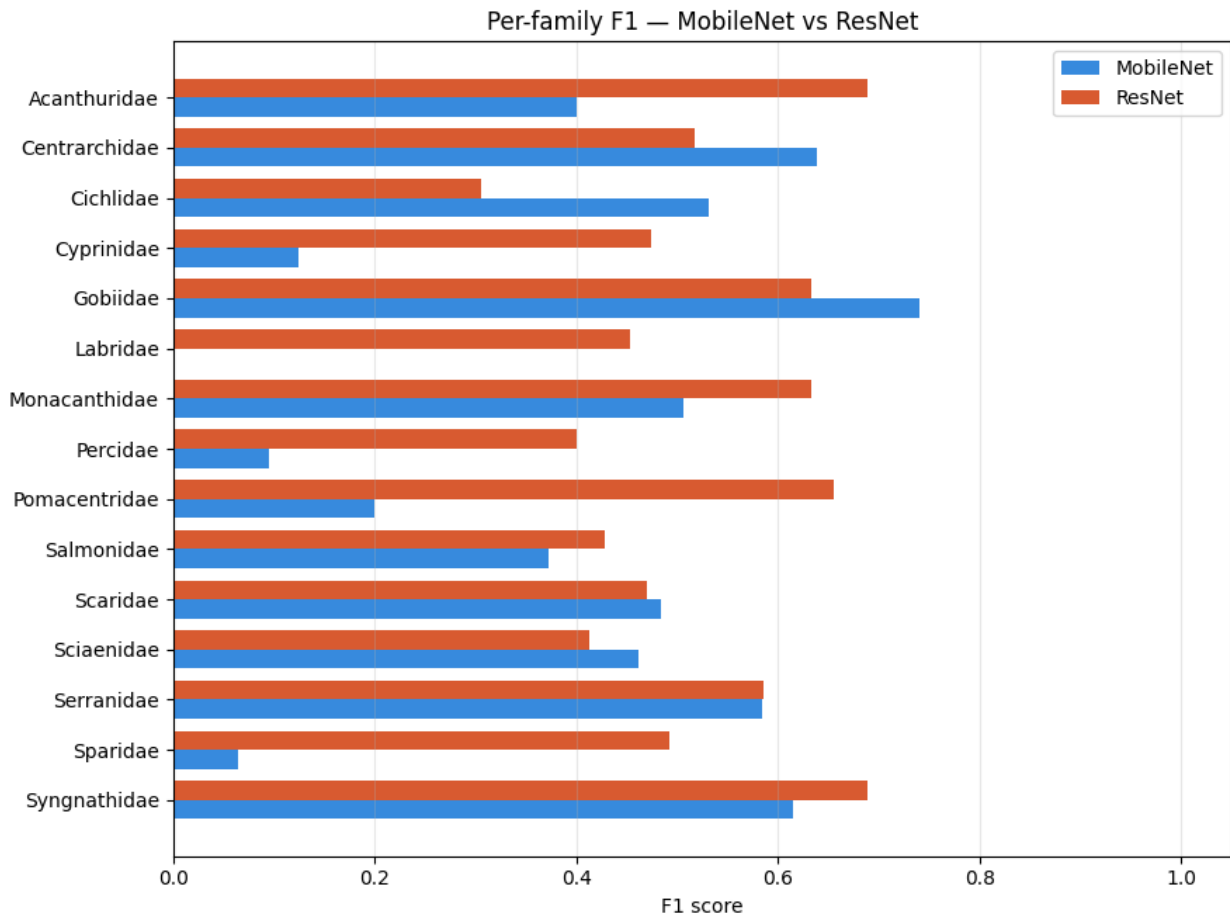
labels = sorted(set(res_mobilenet["y_true"]))
names = [idx_to_family[i] for i in labels]

f1_mob = f1_score(res_mobilenet["y_true"], res_mobilenet["y_pred"],
                  labels=labels, average=None, zero_division=0)
f1_res = f1_score(res_resnet["y_true"], res_resnet["y_pred"],
                  labels=labels, average=None, zero_division=0)

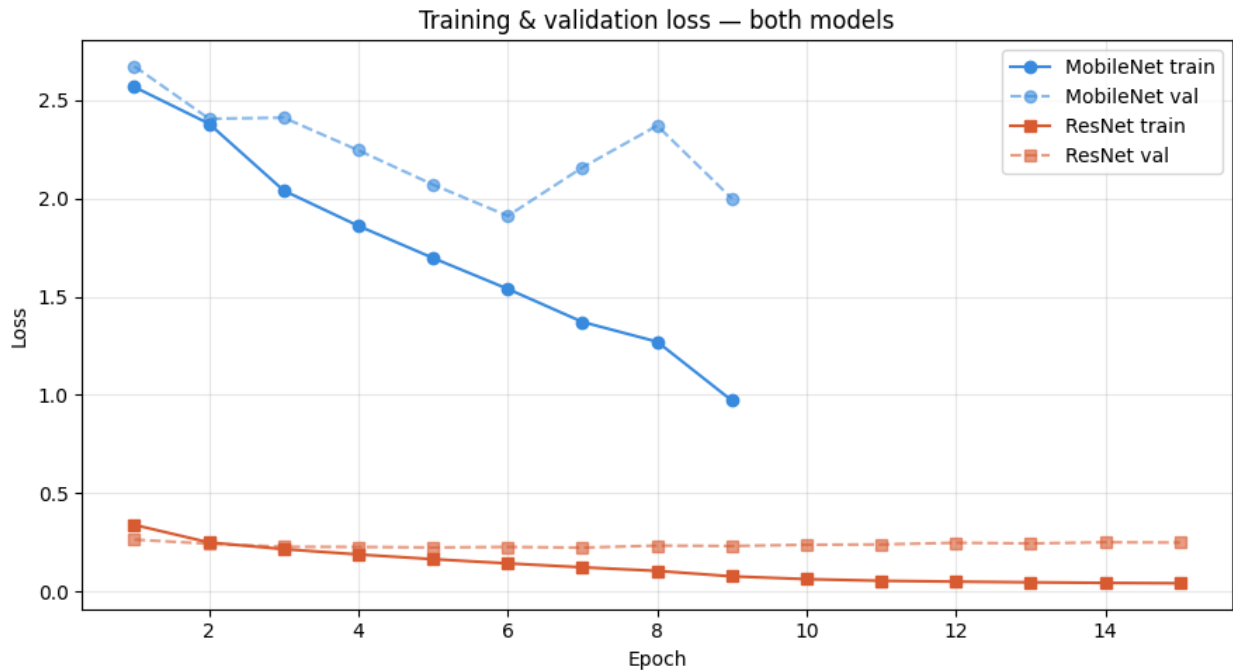
y = np.arange(len(names)); h = 0.38
plt.figure(figsize=(9, max(5, len(names) * 0.45)))
plt.barh(y + h/2, f1_mob, h, label="MobileNet",
         color=COLORS["MobileNet"])
plt.barh(y - h/2, f1_res, h, label="ResNet",
         color=COLORS["ResNet"])
plt.yticks(y, names)
plt.xlabel("F1 score"); plt.xlim(0, 1.05)
plt.title("Per-family F1 – MobileNet vs ResNet")
plt.legend(); plt.grid(axis="x", alpha=0.3)

```

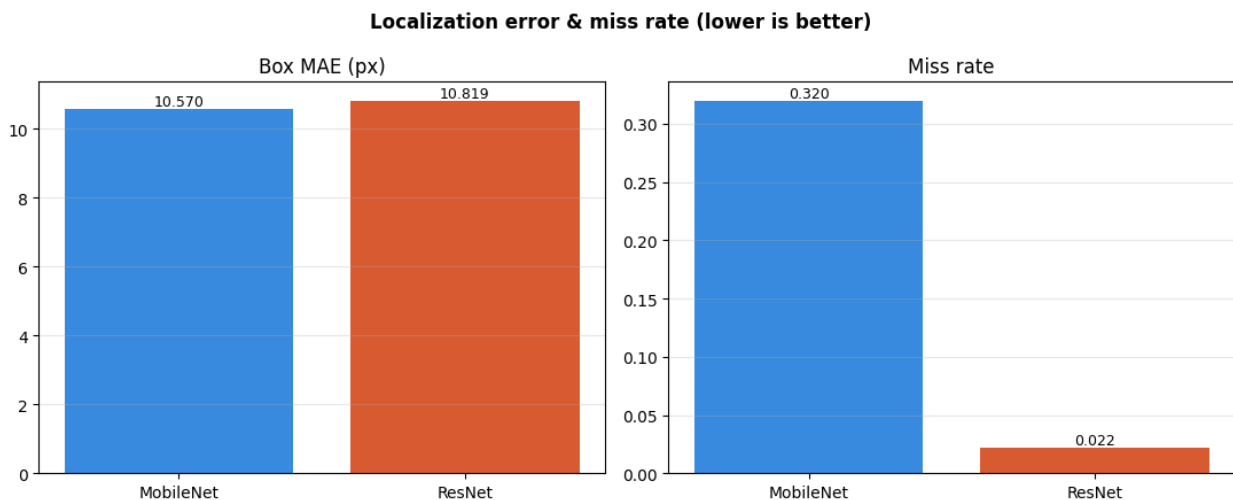
```
plt.gca().invert_yaxis()
plt.tight_layout(); plt.show()
```



```
plt.figure(figsize=(9, 5))
if train_losses_mobilenet:
    e = range(1, len(train_losses_mobilenet) + 1)
    plt.plot(e, train_losses_mobilenet, marker="o",
color=COLORS["MobileNet"], label="MobileNet train")
    plt.plot(e, val_losses_mobilenet, marker="o", ls="--",
color=COLORS["MobileNet"], alpha=0.6, label="MobileNet val")
if train_losses_resnet:
    e = range(1, len(train_losses_resnet) + 1)
    plt.plot(e, train_losses_resnet, marker="s",
color=COLORS["ResNet"], label="ResNet train")
    plt.plot(e, val_losses_resnet, marker="s", ls="--",
color=COLORS["ResNet"], alpha=0.6, label="ResNet val")
plt.xlabel("Epoch"); plt.ylabel("Loss")
plt.title("Training & validation loss – both models")
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```

```
comp = ["Box MAE (px)", "Miss rate"]
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
for ax, metric in zip(axes, comp):
    vals = [overall_metrics(res)[metric] for res in MODELS.values()]
    bars = ax.bar(list(MODELS.keys()), vals, color=[COLORS[n] for n in
MODELS])
    for b, v in zip(bars, vals):
        ax.text(b.get_x() + b.get_width()/2, v, f"{v:.3f}",
ha="center", va="bottom", fontsize=9)
    ax.set_title(metric); ax.grid(axis="y", alpha=0.3)
fig.suptitle("Localization error & miss rate (lower is better)",
fontweight="bold")
plt.tight_layout(); plt.show()
```



Exploring results Further and making conclusion

so now that both models are fully trained and evaluated i can actually compare them properly. a couple of things stand out.

first one: resnet was better and beat mobilenet. resnet landed an accuracy of 0.52 and a macro f1 of 0.49, while mobilenet only managed 0.36 on both. but the number that builds my argument like i mentioned before is the miss rate. mobilenet failed to make any confident prediction (above the 0.3 score threshold) on 144 of the 450 validation images, thats basically a third of them. resnet only missed 10. so most of mobilenets bad score isn't even wrong predictions, its just no predictions at all.

- the precision vs recall split backs up my earlier theory. mobilenet actually has decent precision (0.55, even slightly higher than resnet) but its recall is low (0.34). so when it does commit to a guess its often right, it just stays quiet way too often. this lines up with what i said before, the model is okay at finding the fish but struggles to actually decide which family it belongs to.
- the box localization is tied. mobilenet sits at 10.6px mean error and resnet at 10.8px (on the 224x224 scale). it means the difference between the two models is NOT in how well they draw the box, both are essentially eqwal. the difference is entirely in detecting and classifying the fish. so the backbone is affecting what gets found and named, not where the box goes.
- the noisy background theory still holds. like i mentioned, a lot of these fish blend into their surroundings, and we're training at a much lower resolution than the original images. that makes it hard for the weaker backbone to even detect a fish is there in the first place, which is probably a big reason for those 144 misses with my mobilenet model.
- resnets train loss dropped all the way down to 0.04 but its validation loss bottomed out around epoch 5-7 (0.22) and then slowly went back up, so resnet actually did overfit the small training set. mobilenet on the other hand never really got its train loss down, so it looks more like it underfit, it didn't memorize the data, it just never learned it well in the first place. so the two models are basically failing in opposite ways.

i also want to be honest about something i found in my own code. my early stopping is comparing the wrong loss. resnets check saved a new "best" model every single epoch because it was looking at training loss (which always goes down) instead of validation loss, which means the resnet i actually evaluated is the epoch 15 version, the most overfit one. mobilenet had a similar mixup and restored an early epoch that wasn't its best either. so both of these numbers are probably a little worse than what these models could really do if i picked the checkpoint on validation loss properly. its a fix id make with more time.

Conclusion

the main takeaway is that for this task the backbone really is the deciding factor. given the exact same data, schedule and cpu budget, resnet nearly doubled the usable accuracy and cut the miss rate from 32% down to 2%, while keeping the box accuracy basically identical. so the heavier backbone isn't drawing better boxes, its finding and identifying way more fish.

i first thought the smaller model would be the safer pick and resnet would be the one to overfit. resnet did overfit, but it still generalized better than mobilenet, because mobilenet just didn't have the capacity to fit this fine grained fish family task well in the first place. so here the extra params helped even with the overfitting.

on the deployment side i don't want to oversell mobilenet. its normally the go-to for running on weaker / edge hardware because its light and fast, but in this case the efficiency didn't pay off, the accuracy drop is too big. so my honest recommendation is resnet for this task unless you're really resource constrained.

a few things that probably could've closed the gap, especially for mobilenet:

- data augmentation (horizontal flips + adjusting the bounding boxes, colour jitter) After research i read that this is to give it more variety and fight the memorizing, especially since the images are already noisy with fish blending into the background.
- freezing more of the backbone layers so it keeps more of its pretrained weights.
- ReduceLROnPlateau instead of a fixed step schedule so the learning rate adapts to the validation loss automatically (i mentioned this option in the reading guide too).
- fixing the checkpoint selection so im actually evaluating each models best validation epoch instead of an overfit / early one.

But besides that for this solution Resnet is the clear winner